

Lexicographic Multiobjective Scatter Search for the Optimization of Sequence-Dependent Selective Disassembly Subject to Multiresource Constraints

Xi Wang Guo, Mengchu Zhou¹, Fellow, IEEE, Shixin Liu², and Liang Qi³, Member, IEEE

Abstract—Industrial products' reuse, recovery, and recycling are very important because of their environmental and economic benefits. Effective product disassembly planning methods can improve their recovery efficiency and reduce their bad environmental impact. However, the existing approaches pay little attention to sequence-dependent disassembly with resource constraints, such as limited disassembly operators and tools, which makes the current planning methods ineffective in practice. This paper considers a multiobjective resource-constrained and sequence-dependent disassembly optimization problem with disassembly precedence constraints. Energy consumption is adopted to evaluate the disassembly efficiency. Its use with traditional optimization criterion leads to a novel multiobjective optimization model such that the energy consumption and disassembly time are minimized while disassembly profit is maximized. Since the problem complexity increases with the number of components in a product, a lexicographic multiobjective scatter search (SS) method is proposed to solve the proposed multiobjective optimization problem. Its effectiveness is verified by comparing the results of linear weight SS and genetic algorithms. The results show that it is able to provide a better solution in a short execution time and fulfills the precedence requirement in a product structure and resource constraints.

Index Terms—Disassembly sequence, multiobjective optimization, multiresource constraints, scatter search (SS), sequence-dependent.

Manuscript received June 7, 2018; revised September 27, 2018; accepted February 10, 2019. Date of publication March 27, 2019; date of current version June 16, 2020. This work was supported in part by the National Key Research and Development Program of China under Grant 2017YFB0306400, in part by National Natural Science Foundation of China under Grant 61573089 and Grant 51405075, and in part by the Liaoning Province Doctoral Research Foundation of China under Grant 20175032. This paper was recommended by Associate Editor H. Takagi. (Corresponding author: Mengchu Zhou.)

X. Guo is with the Computer and Communication Engineering College, Liaoning Shihua University, Fushun 113001, China, also with the Key Laboratory of Network Control System, Institute of Automation, Chinese Academy of Sciences, Shenyang 110016, China, and also with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: x.w.guo@163.com).

M. Zhou is with the Institute of Systems Engineering, Macau University of Science and Technology, Macau 999078, China and also with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ 07102 USA (e-mail: zhou@njit.edu).

S. Liu is with the College of Information Science and Engineering, Northeastern University, Shenyang 110819, China (e-mail: sxliu@mail.neu.edu.cn).

L. Qi is with the Department of Computer Science and Technology, Shandong University of Science and Technology, Qingdao 266590, China (e-mail: qiliangskd@163.com).

Color versions of one or more of the figures in this paper are available online at <http://ieeexplore.ieee.org>.

Digital Object Identifier 10.1109/TCYB.2019.2901834

I. INTRODUCTION

PRODUCT recovery enables us to obtain reusable components from discarded or outdated products through remanufacturing and recycling. It can minimize the amount of waste sent to landfills as well as to maximize their reuse value. There are several steps to enhance product recovery. Disassembly is the first essential step for the remanufacturing of various products. It has attracted more and more attention and become an active research area with significant application values.

Disassembly sequence planning (DSP) determines the order in which components are removed from products aiming at minimizing the disassembly time or cost. It has been proved as an NP-hard problem [1]. In the previous research, heuristics and meta-heuristics are used to find near-optimum or optimum solutions and generate cost-effective and feasible disassembly sequences. Heuristics include rule-based recursive method [2] and graph-based heuristic approach [3]. In addition, many meta-heuristics are adopted to optimize a disassembly process, such as genetic algorithms (GAs) [4]–[10]; particle swarm optimization [11]–[16]; ant colony optimization (ACO) [17]–[19]; and greedy randomized adaptive search procedure (GRASP) [20]–[22]. Tseng *et al.* [4] proposed a new block-based GA for DSP based on the Kongar and Gupta's GA and the Dijkstra's algorithms. Go *et al.* [6] presented an optimization model for disassembly sequencing by using GA. Their fitness function in GA is dependent on the increment of disassembly time. Alshibli *et al.* [9] improved a previously proposed GA for disassembly sequencing by utilizing a faster meta-heuristic algorithm. Yeh [14] proposed a simplified swarm optimization method for the update mechanism to revise a self-adaptive parameter control procedure in disassembly sequencing. Li *et al.* [16] developed a swarm optimization algorithm for selective disassembly planning. Wang *et al.* [17] solved the disassembly problem for an automotive engine, which consists of 50 components after the simplification. Malik [18] developed a modified ACO to solve the problem of DSP. If an element in a path cannot be disassembled, this path is marked as an infeasible one. In addition, McGovern and Gupta [19] presented ACO to solve a disassembly sequencing problem. Adenso-Díaz *et al.* [21] developed a bicriteria problem for DSP to fulfill two objectives, where the first one is to minimize the cost of disassembly by keeping it close to a predefined aspiration level, and the other is to remove components

according to a priority list. They propose a GRASP and a path-relinking-based heuristic methodology to solve it.

In order to solve a DSP problem, many researchers propose different approaches. For example, Gao *et al.* [23] proposed a disassembly planning and remanufacturing scheduling method for an integrated flexible remanufacturing system such that the optimal disassembly path with the maximal end-of-life value is obtained. Gungor and Gupta [24] described the uncertainty-related difficulties in DSP. They present a methodology for the disassembly of products with defective components. Brennan *et al.* [25] described a disassembly process of systematic removal of desirable constituent parts from an assembly while ensuring that there is no impairment for the components. Tian *et al.* [26]–[28] proposed a chance-constrained programming approach to determine an optimal disassembly sequence. Tang *et al.* [29], [30] reviewed several methods for modeling and process planning in disassembly. Zussman and Zhou [31] proposed a mathematically sound disassembly Petri net for modeling and adaptive planning of disassembly processes. The planning algorithm guarantees the optimal remanufacturing value when each node's utility function (benefits of a subassembly or part and cost of a disassembly operation) is fixed. The proposed methodology and algorithms are demonstrated through the disassembly of a telephone. Kim *et al.* [32] discussed the automatic control sequence generation for a hybrid disassembly system. Aguinaga *et al.* [33] optimized the selective disassembly by using a random tree-based algorithm. Ren *et al.* [34] improved a previously proposed artificial bee colony model for disassembly sequencing by utilizing a faster metaheuristic algorithm, such as a GA search to obtain the suboptimal solution. Gao *et al.* [35] realized the intelligent decision making of a disassembly process based on fuzzy reasoning Petri nets. Liu *et al.* [36] presented a DSP approach to virtual maintenance of products based on an improved max-min ant system. Tang and Zhou [37] considered the uncertain disassembly time and quality of disassembled subassemblies in a disassembly process. They analyze the expected disassembly cost and expected net profit of product disassembly based on a generic model for a human-in-the-loop disassembly system. They discuss such system where a machine learning approach based on both disassembly Petri net and hybrid Bayesian network is used. In particular, this method models a disassembly process and predicts the outcome of each disassembly action by examining the probabilistic relationships among the different aspects of a disassembly process [38]. Guo *et al.* [39] presented a single-objective disassembly sequence optimization problem with resource constraints. Reveliotis [40] presented the uncertainty management in optimal disassembly planning through learning-based strategies.

From the above discussion, we conclude that prior researches mainly focus on disassembly sequence optimization with a single objective, that is, minimum disassembly time or cost, or maximum profit. In an actual disassembly process of discarded products, the management of components and the use of advanced technologies can protect environment and save energy. A decision maker may also concern an

energy consumption issue when completing a disassembly task because of the pressure of the environment protection. Thus, a multiobjective optimization issue needs to be addressed to minimize the disassembly energy consumption as well as maximize disassembly profit and minimize disassembly time. Note that these objectives may have conflict with each other. Besides, the existing work does not pay much attention to resource constraints, while resource availability should be considered when an optimal disassembly sequence is determined. To deal with this problem, we establish a multiobjective optimization model for a single product disassembly subject to multiresource constraints. This paper proposes a new way to determine the approximate optimal and the optimal disassembly sequence by taking multiple objectives and multiresource and sequence-dependent constraints into full consideration for the first time to our best knowledge. Given many objectives, decision makers or managers set their priority, for example, energy consumption as the most important one, treating profit the second, and time the least important. To solve the problem, we propose to integrate a lexicographic rule in scatter search (SS), which has never been proposed and used before to our best knowledge. This paper has three contributions.

- 1) It proposes a new disassembly model by considering the effect of sequence-dependent and multiresource constraints as well as multiple objectives, including minimum disassembly energy consumption as well as maximum disassembly profit and minimum disassembly time, and allows one to better describe actual conditions.
- 2) It proposes a lexicographic multiobjective SS (LMSS) algorithm to solve the problem. LMSS contains three steps: a) generation of initial solutions; b) combination operator; and c) reference set update. Three kinds of combination operators are proposed: a) path-relink combination operator (PR); b) precedence-preserving crossover combination operator (PX); and c) voting combination operator (VC). We give several cases and compare the performance of different combination operators. The results show that PX is the best one in terms of the efficiency and performance.
- 3) By conducting the experiments on a practical instance and randomly generated instances and comparing LMSS with linear weight SS (LWSS) and GA, this paper shows the effectiveness and feasibility of the proposed algorithm in solving the concerned problem.

The rest of this paper is organized as follows. Section II describes a disassembly problem and establishes its mathematical model. Section III designs an LMSS algorithm. Section IV presents the solutions for several cases and verifies the effectiveness of LMSS. Finally, Section V concludes this paper and describes some future research issues.

II. PROBLEM STATEMENT

A. Problem Statement

A multiobjective multiresource-constrained and sequence-dependent disassembly optimization problem is described as follows: for a product to be disassembled, J is the number of its components, and $j \in \{1, 2, \dots, J\}$ represent the component

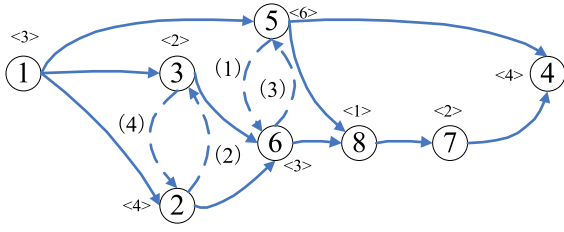


Fig. 1. Priority diagram of a PC [$\textcircled{i}$ = the i th component, $\langle j \rangle$ = the j th resource, and (k) means time increment k].

index. An example of the disassembly of a personal computer (PC) is shown in Fig. 1 [41]. The total number of components is 8. The solid line arrows represent the precedence relations between two components. Note that an arrow from a component to another represents that the latter can be disassembled only if the former one is so. In addition, the disassembly cost and time of two adjacent components (linked by an arrow) would change with a disassembly sequence. The dashed line arrows represent dependency between two components. The component before the dashed arrow hinders the disassembly of the latter. A concept of sequence-dependent time increment of a component [41] is handled in this paper. It is generated by disassembling its following component. Note that components ② and ③ hinder each other. When a component is subject to another sequence-dependent component, its total disassembly time is the sum of its disassembly time and its sequence-dependent time increment. In Fig. 1, the number in parentheses represents sequence-dependent time increments of the component. For instance, disassembling component ② needs 4 s sequence-dependent time increment, which is hindered by component ③. A feasible disassembly sequence should satisfy the following constraints.

- 1) A disassembly process should satisfy precedence relationship constraints among components. For example, in Fig. 1, for a given disassembly sequence $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 7 \rightarrow 4$, component ② can be disassembled before ③, and ③ can be done before ⑥.
- 2) Disassembly of each component may consume multiple resources. A feasible disassembly plan must satisfy resource constraints. In Fig. 1, the number in angle brackets represents the number of resources needed when this component is disassembled. For example, the needed resource quantity is 3 to disassemble component ① as indicated by $\langle 3 \rangle$, and four units of the resources are needed to disassemble ② as indicated by $\langle 4 \rangle$.
- 3) A disassembly sequence-dependent time increment should be considered in a disassembly process. In Fig. 1, for a given disassembly sequence $1 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 7 \rightarrow 4$, ③ is disassembled before ②. The sequence-dependent time increment of ③ is two time units. Similarly, ⑤ is disassembled before ⑥, and its sequence-dependent time increment is three time units.

As shown in Fig. 1, there are different paths to obtain certain components. The aim of this paper is to obtain the best path. Note that there are some features for the problem: 1) a disassembly process is subject to multiresource constraints; 2) there

is a sequence-dependent relationship between components during a disassembly process; 3) it is a complex integer programming problem; and 4) the objective is to minimize energy consumption and time, and maximize disassembly profit. As the number of components and sequence-dependent one's increases, solving the problem becomes much more difficult. Thus, this paper designs LMSS to obtain an approximate solution.

B. Mathematical Model

The notations are summarized as follows.

j, k	Index of a component, where $j, k \in \{1, 2, \dots, J\}$.
J	Number of components composing a product.
C_j	Cost of disassembling j .
R_j	Profit of disassembling j .
c_{jk}	Extra cost to disassemble k after disassembling its previous component j because of the resource change.
p_{jk}	Extra power to disassemble k after disassembling its previous component j because of the resource change.
P_j	Consumed energy to disassemble j .
T_j	Consumed time to disassemble j .
t_{jk}	Extra time of disassembling k after its previous component j as caused by the resource change.
d_{jk}	Sequence-dependent time of disassembling k , which is influenced by component j .
r	Index of a resource type when disassembling a component, where $r \in \{1, 2, \dots, Q_r\}$.
g_{rj}	Number of consumed type- r resources when disassembling j .
r_{rjk}	Number of consumed type- r resources when disassembling k after its previous component j .
Q_r	Maximum number of type- r resources.

Decision Variables:

x_j	$x_j = 1$, if j is disassembled; otherwise, $x_j = 0$.
y_{jk}	$y_{jk} = 1$, if disassembling k after the previous component j adjacent to it; otherwise, $y_{jk} = 0$.

Note that the units of cost and value are RMB and the unit of time is second. By considering three goals of minimizing energy consumption, minimizing disassembly time, and maximizing revenue, we formulate a multiobjective selective disassembly sequence model subject to multiresource constraints as follows:

$$\begin{aligned} \text{Min } Z_1 = & \sum_{j=1}^J \sum_{k=1}^J p_{jk} * t_{jk} * y_{jk} \\ & + \sum_{j=1}^J \left(P_j * T_j + \sum_{k=1}^J p_{kj} * d_{kj} \right) * x_j \end{aligned} \quad (1)$$

$$\text{Max } Z_2 = \sum_{j=1}^J (R_j - C_j) * x_j - \sum_{j=1}^J \sum_{k=1}^J c_{jk} * y_{jk} \quad (2)$$

$$\text{Min } Z_3 = \sum_{j=1}^J \sum_{k=1}^J t_{jk} * y_{jk} + \sum_{j=1}^J \left(T_j + \sum_{k=1}^J d_{kj} \right) * x_j \quad (3)$$

$$\text{s.t. } x_k = \sum_{j=1}^J y_{jk}, \quad k = 1, 2, \dots, J \quad (4)$$

$$\sum_{j=1}^J g_{rj}x_j + \sum_{j=1}^J \sum_{k=1, k \neq j}^J r_{rjk}y_{jk} \leq Q_r r = 1, 2, \dots, R \quad (5)$$

$$\sum_{k=1}^J y_{jk} \leq 1, j = 1, 2, \dots, J \quad (6)$$

$$x_j, y_{kj} \in \{0, 1\}, j, k = 1, 2, \dots, J. \quad (7)$$

Objective function (1) aims to obtain the minimum energy to disassemble a product, where its first item is extra energy, and second one is disassembly energy and sequence-dependent one; (2) seeks the maximum profit to disassemble a product, where its first item is the total profit and the second one is extra one; (3) seeks the minimum disassembly time to disassemble a product, where its first item is extra time and second one is the sequence-dependent time. Equation (4) gives the relationship between two decision variables (each component can be disassembled at most once); (5) poses the resource constraints to disassemble a product; constraint (6) assures that a disassembly operation can be performed only once; and (7) restricts that decision variables to 1 or 0.

III. SCATTER SEARCH ALGORITHM DESIGN

As a heuristic algorithm, SS has been proved to solve many complex combination optimization problems [43]. It contains five steps: 1) generation of diverse initial solutions; 2) generation of a reference set; 3) producing a subset; 4) solutions combination; and 5) reference set updating. Note that its realization way is different for different problems. This paper adapts a basic SS algorithm to solve the optimization problem. In order to generate high-quality solutions, LMSS and LWSS are adopted to improve the quality of solutions and use the maximum iteration as the stopping criterion. In LMSS, each individual represents a solution and a population is used for finding the near-optimal solution of the problem. The main steps of LMSS are shown in Fig. 2.

A. Encoding and Decoding of Solutions

The concerned problem is a sorting and combination optimization problem. According to its features, a double-link list structure is adopted as its encoding form, in which an individual is expressed by $I = (s^1, s^2)$, where $s^1 = (s_1, \dots, s_i, \dots, s_J)$ represents a component sequence, and $s^2 = (x_1, \dots, x_i, \dots, x_J)$ represents whether components have been disassembled. By taking a product disassembly in Fig. 1 as an example, we present its encoding process for the optimization problem. Note that the problem parameters of each component are shown in Table I, including the number of available disassembly resources, disassembly cost, disassembly time, the value of its component, and disassembly energy. Extra cost to disassemble each component is set in Table II, and the extra time of disassembling each component in this sequence is set in Table III. In addition, the extra energy of disassembling each component in this sequence is set in Table IV.

Let us randomly select a feasible individual $I = (s^1, s^2)$, as shown in Table V. Then, the corresponding feasible disassembly sequence of this individual is $s^1 =$

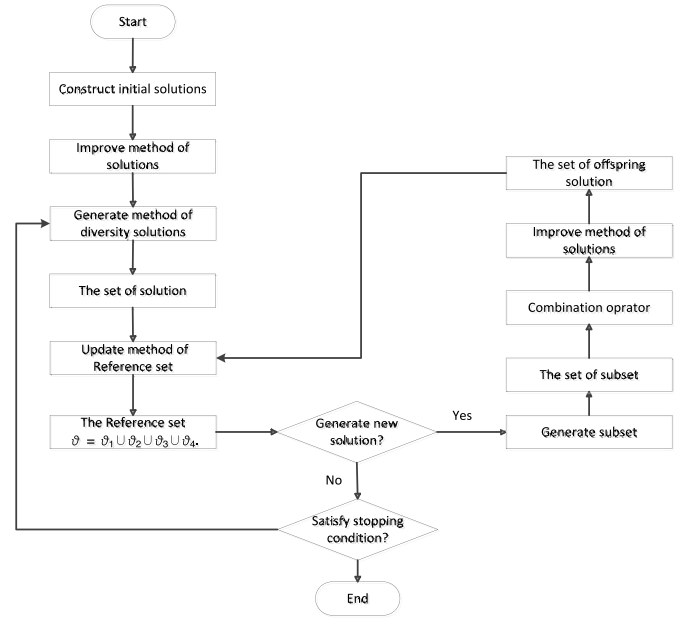


Fig. 2. Flowchart of LMSS.

(1, 2, 3, 5, 6, 8, 7, 4) and $s^2 = (1, 1, 1, 1, 1, 1, 1, 1)$. We can see that $x_1, x_2, x_3, x_5, x_6, x_8, x_7$, and x_4 in s^2 are 1, which represents that operations $s_1, s_2, s_3, s_5, s_6, s_8, s_7$, and s_4 in s^1 is disassembled at that time. Thus, the real disassembly sequence is $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 7 \rightarrow 4$. The value of c_{jk}, t_{jk} , and p_{jk} are shown in Tables II–IV. Based on (1)–(3), we obtain the results: objective function Z_1 is 765 and Z_2 is 30. Also, the total consumed time can be obtained by objective function (3) with the value of $Z_3 = 114$. The above description is an evaluation process for this individual and its result is shown in Table V. Similarly, the objective function values can be obtained from randomly generated feasible individuals.

B. Generation Method of Initial Solution Set Ω

Usually, the number of solutions increases drastically with the number of components. Some solutions among them may not satisfy a precedence relationship constraint. In order to solve this problem, this paper adopts a double-link list structure as an encoding form. The procedure to produce a set of initial solutions is presented in Algorithm 1. It improves the efficiency of the algorithm because it is easy to get a feasible solution. Otherwise, the algorithm tend to discard thousands of infeasible solutions in order to obtain one feasible solution, severely affecting the efficiency of the algorithm.

C. Generation of Reference Set

A reference set, denoted as θ , is used to store high-quality and diversified solutions. θ is composed of mutually exclusive subsets. Because of three optimization objectives, three mutually exclusive subsets are designed in θ . Its size is denoted by $|\theta|$. Let θ_i represents the subset of solutions with high quality according to the i th objective $i \in \{1, 2, 3\}$, and θ_4 be the subset of highly diverse solutions. High-quality and diversified subsets are selected from initial solution set to form the initial

TABLE I
PARAMETERS IN EXAMPLE 1 ($Q_r = 50$)

index	1	2	3	4	5	6	7	8
T_j	9	3	9	8	17	6	7	8
R_j	17	15	19	16	18	17	15	15
C_j	8	6	7	7	9	8	6	7
g_{rj}	5	4	2	4	3	2	4	1
P_j	7	4	9	5	10	9	7	6

TABLE II
EXTRA COST IN EXAMPLE 1 (UNIT: RMB)

c_{jk}	1	2	3	4	5	6	7	8
1	0	5	3	0	6	0	0	0
2	0	0	9	0	8	7	0	0
3	0	4	0	0	9	8	0	0
4	0	0	0	0	0	0	0	0
5	0	6	5	4	0	6	0	5
6	0	0	0	0	5	0	0	6
7	0	0	0	5	0	0	0	0
8	0	0	0	0	0	0	4	0

TABLE III
EXTRA TIME IN EXAMPLE 1 (UNIT: SECOND)

t_{jk}	1	2	3	4	5	6	7	8
1	0	4	2	0	5	0	0	0
2	0	0	8	0	7	6	0	0
3	0	3	0	0	8	7	0	0
4	0	0	0	0	0	0	0	0
5	0	5	4	3	0	5	0	4
6	0	0	0	0	4	0	0	5
7	0	0	0	4	0	0	0	0
8	0	0	0	0	0	0	3	0

TABLE IV
EXTRA ENERGY IN EXAMPLE 1 (UNIT: JOULE)

p_{jk}	1	2	3	4	5	6	7	8
1	0	5	4	0	4	0	0	0
2	0	0	7	0	5	5	0	0
3	0	4	0	0	8	6	0	0
4	0	0	0	0	0	0	0	0
5	0	3	2	6	0	7	0	4
6	0	0	0	0	3	0	0	5
7	0	0	0	3	0	0	0	0
8	0	0	0	0	0	0	2	0

TABLE V
CODING AND THE OBJECTIVE FUNCTION OF AN INDIVIDUAL

s^1	1	2	3	5	6	8	7	4
s^2	1	1	1	1	1	1	1	1
Objectives	$Z_1=765, Z_2=30$ and $Z_3=114$							

reference set. The distance between two solutions I_i and I_k is defined as

$$D(I_i, I_k) = \sum_{j=1}^J \left| p_j(s_i^1) - p_j(s_k^1) \right| + \sum_{j=1}^J \left| h_j(s_i^2) - h_j(s_k^2) \right| + \sum_{j=1}^J \sum_{k=1}^J \left| y_{jk}^1 - y_{jk}^2 \right| \quad (8)$$

where s_i^1 and s_k^1 represent the corresponding component in individuals I_i and I_k , respectively. $p_j(s_i^1)$ and $p_j(s_k^1)$ represent

the position of component j in a disassembly sequence s_i^1 and s_k^1 , respectively. $h_j(s_i^2)$ and $h_j(s_k^2)$ represent the value of binary variables in s_i^2 and s_k^2 , respectively. y_{jk}^1 and y_{jk}^2 represent the values of binary variables corresponding to s_i^1 and s_k^1 , respectively. In addition, $|\theta_4|$ solutions with the maximum distance $D(I_i, \theta)$ from the current reference set are selected from the initial solution set Ω

$$D(I_i, \theta) = \min\{D(I_i, I_k) | I_k \in \theta\} \quad (9)$$

where θ represents the current reference set and $D(I_i, \theta)$ represents the distance between I_i and θ . Algorithm 2 can be used

Algorithm 1 Generation of Initial Solution Set Ω

Input: $I = (s^1, s^2)$
Output: Ω
Procedure $\Omega(I = (s^1, s^2), \Omega)$

- (1) While ($i \leq \Omega_S$) do
- (2) Randomly generate the first section of coding that satisfies a priority relationship, i.e., $s^1 = (s_1, \dots, s_i, \dots, s_J)$;
- (3) Randomly generate the second section of coding, i.e., $s^2 = (x_1, \dots, x_i, \dots, x_J)$;
- (4) Construct the initial solution $I = (s^1, s^2)$;
- (5) If (I is feasible) then
- (6) Insert I to the initial solution set Ω ;
- (7) $i++$;
- (8) Else
- (9) While ($j \leq J$) do
- (10) If($s^2(j) == 1$) then
- (11) Assign precursor node "1";
- (12) Else
- (13) Set it "0";
- (14) $j++$;
- (15) End While;
- (16) End While;

Algorithm 2 Generation of Reference Set

Input: Ω
Output: $\theta, \theta_1, \theta_2, \theta_3$, and θ_4
Procedure $\theta(\Omega, \theta, \theta_1, \theta_2, \theta_3, \theta_4)$

- (1) Select $|\theta_1|$ best individuals from initial solution set Ω according to their Z_1 value calculated via (1).
- (2) Select $|\theta_2|$ best individuals from initial solution set Ω according to their Z_2 value calculated via (2).
- (3) Select $|\theta_3|$ best individuals from initial solution set Ω according to their Z_3 value calculated via (3).
- (4) Build θ_1 with these $|\theta_1|$ individuals
- (5) Build θ_2 with these $|\theta_2|$ individuals
- (6) Build θ_3 with these $|\theta_3|$ individuals
- (7) Select $|\theta_4|$ individuals with maximal D, calculated by (7).
- (8) Set $\theta = \theta_1 \cup \theta_2 \cup \theta_3 \cup \theta_4$.

to generate a reference set. It can dynamically track the current search status to adjust its search strategy as the memory ability of its reference set, which can reduce the randomness of a search process, and adopt some systematic ways to build a new solution.

D. Combination Operator of Subsets and Solutions

A combination operator combines high-quality solutions and diverse ones into new solutions. Its design guarantees not only high-quality solutions but also the high diversity of solutions. According to the double-link list structure of the individual and precedence constraints, we employ the precedence-preserving crossover (PX) and path_relink (PR) as combination operators [42]. A PX and PR combined operation is described in [39]. In order to verify the influence of different combination operators on algorithm performance, we adopt VC in Algorithm 3. It can quickly obtain a satisfactory solution and avoid prematurely falling into a local optimal solution. Note that $I_1 = (s_1^1, s_1^2)$ and $I_2 = (s_2^1, s_2^2)$ represent the VC operation of two participating individuals, respectively. $I_c = (s_c^1, s_c^2)$ represents an offspring individual resulting from the combination. $s_1^1(i)$, $s_2^1(i)$, and $s_c^1(i)$ represent the corresponding disassembly operation of position i in s_1^1 , s_2^1 , and s_c^1 , respectively. Similarly, $s_1^2(i)$, $s_2^2(i)$, and $s_c^2(i)$ represent the corresponding value of binary variables of position i in s_1^2 , s_2^2 , and s_c^2 , respectively.

Algorithm 3 Process of VC Operation

Input: $I_1 = (s_1^1, s_1^2), I_2 = (s_2^1, s_2^2)$
Output: $I_c = (s_c^1, s_c^2)$
Procedure VC($I_1 = (s_1^1, s_1^2), I_2 = (s_2^1, s_2^2), I_c = (s_c^1, s_c^2)$)

- (1) o_1 is Z_1 value according to I_1 calculated via (1).
- (2) o_2 is Z_1 value according to I_2 calculated via (1).
- (3) v_1 (i) record the position of every component i in I_1 ;
- (4) v_2 (i) record the position of every component i in I_2 ;
- (5) While ($i \leq n$) do
- (6) If ($v_1(i) > v_2(i)$) then $a = s_1^1(i), b = s_2^2(i)$;
- (7) Else If ($v_1(i) < v_2(i)$) then $a = s_2^1(i), b = s_2^2(i)$;
- (8) Else if($o_1(i) > o_2(i)$) then $a = s_1^1(i), b = s_1^2(i)$;
- (9) Else $a = s_2^1(i), b = s_2^2(i)$;
- (10) $s_c^1(i) = a$;
- (11) $s_c^2(i) = b$;
- (12) $i++$;
- (13) End While;

Algorithm 4 Process of Reference Set Update

Input: A feasible solution Γ
Output: θ
Reference Set Update (Γ, θ)

- (1) Use Algorithm 2 to construct a reference set θ
- (2) For (each new solution Γ) {
- (3) If (test $Z_1(\Gamma)$ is smaller than objective function value in θ_1, θ_2 and θ_3)
- (4) Insert Γ in $\theta_1, \theta_2, \theta_3$;
- (5) Else
- (6) If (test $Z_2(\Gamma)$ is bigger than the objective function value in θ_1, θ_2 and θ_3)
- (7) Insert Γ in θ_1, θ_2 and θ_3 ;
- (8) Else
- (9) If (test $Z_3(\Gamma)$ is smaller than objective function value in θ_1, θ_2 and θ_3)
- (10) Insert Γ in θ_1, θ_2 and θ_3 ;
- (11) Else
- (12) If ($D(\Gamma, \theta)$) is bigger than distances in θ_4)
- (13) Insert Γ in θ_4 ;
- (14) Else
- (15) Delete Γ ;
- (16) EndIf
- (17) EndIf
- (18) EndIf
- (19) EndIf
- (20) EndFor

E. Update Reference Set

In SS, a reference set updating method uses improved solutions to update the reference set, and always keeps high-quality solutions and highly diverse ones. In order to derive improved solutions, we integrate a lexicographic rule to update the reference set. Its process is given in Algorithm 4. By Algorithm 4, we know that a new individual is accepted only if it is feasible and its corresponding objective function value is improved over the original one. Note that, in this paper, facing many objectives, decision makers or managers have their priority, e.g., LMSS, LWSS, and GA regard Z_1 (energy consumption) as the most important optimization objective, then Z_2 (profit) as the second one, and Z_3 (disassembly time) as the last one.

F. Termination Criterion

SS adopts the maximum iteration count as a termination criterion, that is, when it is reached, output the solution and terminate the algorithm.

Algorithm 5 Process of Scatter Search

Input: Γ

Output: p^*

Procedure scatter search (Γ, p^*)

(1) Use Algorithm 1 to construct a solution set Ω , $|\Omega| = \text{PSize}$;

(2) While (*no stopping criteria are met*) do

(3) Use Algorithm 2 to generate the initial reference set θ . Set

```
NewSolutions = True;
```

(4) While (*NewSolutions*) Do

(5) Generate a subset from current reference set θ , and make

```
NewSolutions = False;
```

(6) Apply Algorithm 3 to combine solutions in each subset;

(7) Apply Algorithm 4 to update θ ;

(8) If (θ is updated) then

```

(9)      Set NewSolutions = True;

```

(10) End if

(11) End While;

(12) Go to Line (1);

(13) End While;

G. SS Procedure

Based on the above discussion, the SS procedure is given in Algorithm 5. The theory and convergence of such SS procedure can be referred to [44]–[48].

IV. EXPERIMENTAL RESULTS AND ANALYSIS

To test the efficiency of LMSS, a product example from [41] as shown in Fig. 3 is given. It contains 25 components. According to the problem characteristics, parameters are set as shown in Table VI. Meanwhile, the value of extra cost, extra time, and extra energy to perform each operation is ranged from 1 to 10. The total quantity of available disassembly resources Q_r is 80. In addition, the algorithm runs on Intel I5 (2.30 GHz/4.0G RAM) PC with a Windows 7 operating system and it is implemented by JAVA programming language. After LMSS is executed, the results are shown in Table VII. The near-optimal disassembly sequence is $1 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 8 \rightarrow 9 \rightarrow 13 \rightarrow 7 \rightarrow 14 \rightarrow 15 \rightarrow 16 \rightarrow 18 \rightarrow 17 \rightarrow 21 \rightarrow 20$, and the objective function Z , that is, energy consumption equals 1300 J, disassembly profit is 85 RMB, and disassembly time is 164 s. The results show that the proposed method can effectively solve a disassembly planning problem.

To further show the efficiency of the proposed algorithms, three kinds of combination operators are used in LMSS, that is, LMSS_PR, LMSS_PX, and LMSS_VC. Comparisons are done for seven products containing 20, 30, 40 50, 60, 80, and 100 disassembly components, respectively. Also, the total number of available resource is $Q_r = 500$. Other parameters are set as shown in Table VIII. In addition, in order to have a fair comparison, the same encoding method, decoding rules, and reference set update method are used. The third group simulation experiments of each seven products run ten times and the results are shown in Table IX. Note that aZ_1 , aZ_2 , and aZ_3 represent the average results of Z_1 , Z_2 , and Z_3 when running ten times, respectively. SD_1 , SD_2 , and SD_3 , respectively, represent the average standard deviations of Z_1 , Z_2 , and Z_3 when running ten times, and a CPU represents average run time when running ten times.

From Table IX, we can obtain the following conclusions.

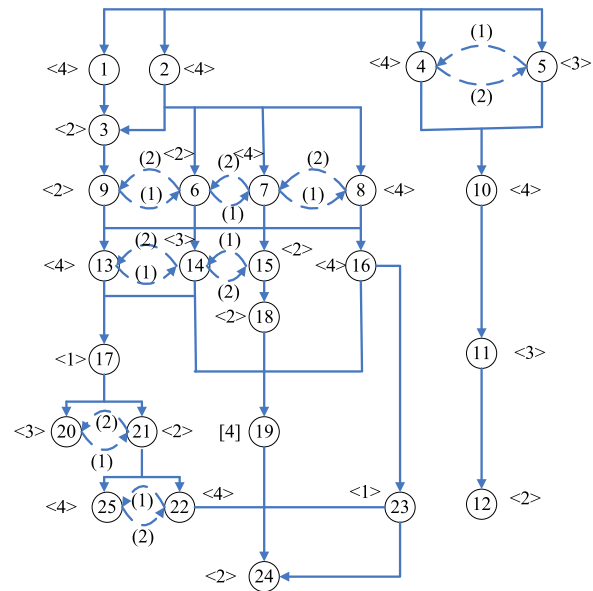


Fig. 3. Structure diagram of a telephone to be disassembled.

- 1) Three kinds of LMSS, that is, LMSS_PR, LMSS_PX, and LMSS_VC can successfully solve each case and obtain the satisfactory solutions. This indicates that the proposed algorithms are feasible to solve the disassembly model with multiresource constraints.
- 2) From the efficiency point of view, SS_PX is the best, and LMSS_VC is better than LMSS_PR. In addition, we can see from Table IX that running time of LMSS_PR runs longer than that of SS_PX and LMSS_VC.

To test the efficiency of LMSS, a linear weight algorithm (LWSS) [39] is used. To have a fair comparison, the same coding method, decoding rules, combination operator (PX), and reference set update method are adopted when running the simulations. The work [39] does not consider resource constraints to solve a completely disassembly sequence optimization problem. It proposes a linear weight method to transform a multiobjective problem into a single objective problem. Note that in the test process, the algorithm in [39] is advised to solve the objective optimization problem in this paper, that is, three objectives Z_1 , Z_2 , and Z_3 are transformed into a signal one $Z = w_1Z_1 + w_2Z_2 + w_3Z_3$. Two algorithms are compared and parameters are set as follows: the maximum iteration count is 100 times, the size of initial population is $\Omega_S = 100$, the size of reference set is $|\theta| = 20$, and $|\theta_1| = |\theta_2| = 10$ from [39]; $|\theta_1| = |\theta_2| = |\theta_3| = 5$, $|\theta_4| = 10$ in the algorithm. Meanwhile, in order to further test the efficiency of LMSS, a traditional method GA [46] is used. It adopts the same encoding method and decoding rules. The algorithm in [46] is used to solve the problem in this paper. The roulette choosing method is used as a selective operator and PX is used as a crossover combination operator. Parameters of GA are set as follows: $\Omega_S = 100$, $p_c = 0.6$, $p_m = 0.005$, and $\text{Gen} = 100$. The results by adopting the method in [39] and [46] are as shown in Table X. We have conducted some preliminary experiments about different parameter combinations. The parameters of

TABLE VI
PROBLEM PARAMETERS OF COMPONENTS

Component index	1	2	3	4	5	6	7	8	9	10	11	12	13
T_j	3	2	3	10	10	15	15	15	15	2	2	2	2
R_j	13	15	19	16	18	17	15	15	19	16	18	17	15
C_j	5	6	7	7	9	8	6	6	7	7	9	8	6
P_j	3	2	3	10	10	15	15	15	15	2	2	2	2
Component index	14	15	16	17	18	19	20	21	22	23	24	25	
T_j	2	2	2	2	3	18	5	1	5	15	2	2	
R_j	14	12	16	15	17	13	14	18	15	16	14	13	
C_j	3	5	4	5	7	5	7	8	4	5	7	3	
P_j	2	2	2	2	3	18	5	1	5	15	2	2	

TABLE VII
CODING AND THE OBJECTIVE FUNCTION OF AN INDIVIDUAL

s^1	1	2	4	6	3	8	9	5	13	7	14	15	16	18	17	21	22	10	11	20	19	23	24	12	25
s^2	1	1	0	1	1	1	1	0	1	1	1	1	1	1	1	1	0	0	0	1	0	0	0	0	0
Objectives	$Z_1=1300, Z_2=85$ and $Z_3=164$																								

TABLE VIII
DIFFERENT PARAMETERS AND RANGE OF VALUE

parameters	r_{rjk}	s_{rj}	R_j	T_j	C_j	t_{jk}	d_{jk}	c_{jk}	p_{jk}	P_j
Range	[1,4]	[1,4]	[1,100]	[1,10]	[1,20]	[1,10]	[1,10]	[1,10]	[1,10]	[1,10]

TABLE IX
RUNNING RESULTS

	LMSS_PR							LMSS_PX							LMSS_VC						
	aZ ₁	SD ₁	aZ ₂	SD ₂	aZ ₃	SD ₃	aCPU	aZ ₁	SD ₁	aZ ₂	SD ₂	aZ ₃	SD ₃	aCPU	aZ ₁	SD ₁	aZ ₂	SD ₂	aZ ₃	SD ₃	aCPU
20	369	37.6	1148	119.68	64	6.97	1086	354	28.32	1034	83.72	60	5.87	404	363	33.76	1037	93.52	72	6.46	454
30	473	48.25	1251	196.99	66	6.52	2043	423	33.49	1478	115.24	71	5.91	599	464	41.49	1424	128.33	67	6.53	606
40	1035	102.87	2968	297.63	102	10.43	3752	964	77.79	2806	225.45	114	9.12	804	980	88.67	2904	261.84	105	9.53	816
50	1669	139.33	4585	456.23	136	13.81	4982	1607	128.85	4581	367.66	140	12.14	1034	1652	149.19	4537	408.01	137	12.47	985
60	2399	224.81	6605	623.09	174	18.12	7256	2355	189.82	7010	560.27	179	14.32	1042	2347	216.75	6664	599.05	179	16.69	1092
80	4742	473.01	13827	1290.78	264	19.35	11832	4586	366.12	13596	1087.13	244	20.83	1800	4593	413.37	13535	1218.15	241	21.89	1503
100	7554	766.72	21419	2032.27	328	34.45	15604	7261	580.27	21555	1750.75	335	26.83	2589	7339	660.79	20534	1848.58	329	28.93	2615

TABLE X
COMPARISON OF LMSS, LWSS, AND GA ($w_1 = 0.6$, $w_2 = 0.2$, AND $w_3 = 0.2$)

n	Algorithm	Iteration count/n	aZ ₁	SD ₁	aZ ₂	SD ₂	aZ ₃	SD ₃	aCPU
20	LMSS	100	354	28.32	1034	83.72	60	5.87	404
	LWSS	100	1124	101.40	3397	305.83	75	6.62	184
	GA	100	1231	110.61	1403	126.93	80	7.19	1958
30	LMSS	100	473	33.49	1478	115.24	71	5.91	599
	LWSS	100	2275	204.78	6679	601.96	119	10.41	278
	GA	100	2363	212.51	3331	299.64	157	14.74	2842
40	LMSS	100	984	77.79	2806	225.45	114	9.12	804
	LWSS	100	1844	167.42	5803	522.65	169	15.49	448
	GA	100	2117	183.24	5977	537.74	202	18.42	6066
50	LMSS	100	1607	128.85	4581	367.66	140	12.14	1034
	LWSS	100	3092	278.37	9351	844.75	218	19.10	592
	GA	100	3493	311.53	9128	821.78	257	23.88	7944
60	LMSS	100	2455	189.82	7010	560.27	179	14.32	1042
	LWSS	100	4490	401.36	12998	1169.79	267	24.96	730
	GA	100	4763	428.60	13762	1238.53	393	35.42	10521
80	LMSS	100	4596	366.12	13596	1087.13	244	20.83	1800
	LWSS	100	6613	595.82	25430	2228.47	410	36.94	1256
	GA	100	6851	616.42	27915	2512.31	435	39.74	17795
100	LMSS	100	7461	580.27	21555	1750.75	335	26.83	2589
	LWSS	100	7737	696.73	37483	3373.82	417	37.13	2078
	GA	100	7865	707.39	34875	3138.70	462	41.58	25854

LMSS and GA are selected based on many experiments by ourselves but not based on any references. Suitable values for those parameters enhance the effectiveness of SS and GA. We have obtained better performance by validating all

test cases by using these values in Tables IX and X in Section IV. Note that LMSS regards Z_1 as the most important optimization objective, then Z_2 as the second one, and Z_3 as the last one.

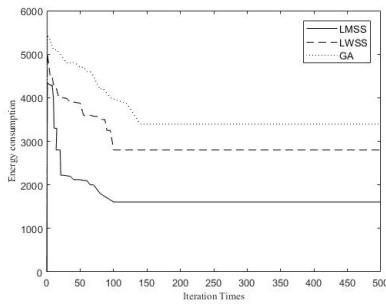


Fig. 4. Convergence curve.

From Tables IX and X, the following conclusions can be drawn.

- 1) Our proposed algorithm can obtain a near-optimal solution with the same number of iterations. Given the same parameters, the objective value rises with the problem scale.
- 2) From the statistical point of view, LMSS can obtain better solutions than LWSS and GA. As can be seen in Table X, LWSS or GA has better results regarding profit, but LWSS or GA has more energy consumption. LMSS can obtain the least energy consumption than LWSS and GA.
- 3) From the running time point of view, LMSS costs the smaller time to obtain solutions than LWSS and GA.
- 4) Three algorithms' running time rises as the number of components increases. In summary, the proposed algorithm is feasible and efficient to solve our concerned optimization problem.

To further test the proposed method, we compare the convergence of LMSS, LWSS, and GA with $w_1 = 0.6$, $w_2 = 0.2$, and $w_3 = 0.2$. The convergence curves are obtained by running the experiment with a product containing 50 components as shown in Fig. 4. Note that Fig. 4 illustrates the tendency of the energy consumption value. It shows that LMSS performs better in convergence and optimality. Note that we have also tried other combinations, e.g., weighting vector $w = (0.7, 0.2, 0.1)$, $(0.5, 0.3, 0.2)$, and $(0.8, 0.1, 0.1)$ and we can get the same conclusions. Because of the limited space, we do not give other convergence curves in this paper. Thus, $w = (0.6, 0.2, 0.2)$ is chosen to show the convergence of LMSS. Besides, we have analyzed its time complexity, which is $O(n^2)$.

The fewer resources lead to the fewer feasible solutions and the smaller solution space, thus limiting disassembly choices; and vice versa. Because of the space limit, we do not discuss the relations among energy consumption, resources, and disassembly decisions.

V. CONCLUSION

This paper addresses a multiobjective resource-constrained selective disassembly sequence optimization problem. Its goal is to minimize the energy consumption and disassembly time, and maximize the disassembly profit. An LMSS algorithm is designed to solve the proposed model, which includes a generation method of diverse initial solutions, a method of reference set update, a generation method of subsets, and a combination

method of subsets. Three different combination operators are designed. The experimental results show that the efficiency of precedence-preserving crossover combination operator is better than that of path-relink and voting combination operator. In addition, the proposed algorithm can solve the optimization problem effectively and efficiently, and generate near-optimal solutions. To show the superiority of the proposed approach in terms of objective function values and CPU times, it is compared with LWSS and GA algorithms. The obtained results indicate that LMSS performs the best. It also shows that LMSS with an appropriate algorithm can be used to solve the concerned problem. Moreover, it can provide solutions in a short time. It can be used to guide users to make better decisions to disassemble a product in the best sequence.

The proposed LMSS can be applied to disassemble a large-scale product with more components, e.g., cars, engines, televisions, and refrigerators. Future work should consider new mathematical formulations developed for different disassembly problems. Finally, the proposed method can be hybridized with other heuristic and meta-heuristic algorithms, such as GA, beam search, and ACO to improve the quality of solutions [47]–[51] and discuss the relation among energy consumption, the amount of resources, and disassembly decisions.

REFERENCES

- [1] A. J. D. Lambert, "Disassembly sequencing: A survey," *Int. J. Prod. Res.*, vol. 41, no. 16, pp. 3721–3759, 2003.
- [2] S. S. Smith and W.-H. Chen, "Rule-based recursive selective disassembly sequence planning for green design," *Adv. Eng. Informat.*, vol. 25, no. 1, pp. 77–87, 2011.
- [3] T. C. Kuo, H. C. Zhang, and S. H. Huang, "Disassembly analysis for electromechanical products: A graph-based heuristic approach," *Int. J. Prod. Res.*, vol. 38, no. 5, pp. 993–1007, 2000.
- [4] H.-E. Tseng, C.-C. Chang, S.-C. Lee, and Y.-M. Huang, "A block-based genetic algorithm for disassembly sequence planning," *Expert Syst. Appl.*, vol. 96, no. 15, pp. 492–505, Apr. 2018.
- [5] F. Giudice and G. Fargione, "Disassembly planning of mechanical systems for service and recovery: A genetic algorithms based approach," *J. Intell. Manuf.*, vol. 18, no. 3, pp. 313–329, 2007.
- [6] T. F. Go, D. A. Wahab, M. N. A. Rahman, R. Ramli, and A. Hussain, "Genetically optimised disassembly sequence for automotive component reuse," *Expert. Syst. Appl.*, vol. 39, no. 5, pp. 5409–5417, 2012.
- [7] W. Hui, X. Dong, and D. Guanghong, "A genetic algorithm for product disassembly sequence planning," *Neurocomputing*, vol. 71, nos. 13–15, pp. 2720–2726, 2008.
- [8] J. R. Li, L. P. Khoo, and S. B. Tor, "A novel representation scheme for disassembly sequence planning," *Int. J. Adv. Manuf. Technol.*, vol. 20, no. 8, pp. 621–630, 2002.
- [9] M. E. Alshibli, A. El Sayed, E. Kongar, T. M. Sobh, and S. M. Gupta, "Disassembly sequencing using Tabu search," *J. Intell. Robot. Syst.*, vol. 82, no. 1, pp. 69–79, 2016.
- [10] J. R. Li, L. P. Khoo, and S. B. Tor, "An object-oriented intelligent disassembly sequence planner for maintenance," *Comput. Ind.*, vol. 56, no. 7, pp. 699–718, 2005.
- [11] Y.-J. Tseng, F.-Y. Yu, and F.-Y. Huang, "A green assembly sequence planning model with a closed-loop assembly and disassembly sequence planning using a particle swarm optimization method," *Int. J. Adv. Manuf. Technol.*, vol. 57, nos. 9–12, pp. 1183–1197, 2011.
- [12] Y. Wang and J. H. Liu, "Chaotic particle swarm optimization for assembly sequence planning," *Robot. Comput. Integr. Manuf.*, vol. 26, no. 2, pp. 212–222, 2010.
- [13] H. Lv and C. Lu, "An assembly sequence planning approach with a discrete particle swarm optimization algorithm," *Int. J. Adv. Manuf. Technol.*, vol. 50, nos. 5–8, pp. 761–770, 2010.
- [14] W.-C. Yeh, "Optimization of the disassembly sequencing problem on the basis of self-adaptive simplified swarm optimization," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 42, no. 1, pp. 250–261, Jan. 2012.

- [15] W.-C. Yeh, "Simplified swarm optimization in disassembly sequencing problems with learning effects," *Comput. Oper. Res.*, vol. 39, no. 9, pp. 2168–2177, 2012.
- [16] W. D. Li, K. Xia, L. Gao, and K. M. Chao, "Selective disassembly planning for waste electrical and electronic equipment with case studies on liquid crystal displays," *Robot. Comput. Integr. Manuf.*, vol. 29, no. 4, pp. 248–260, 2013.
- [17] J. F. Wang, J. H. Liu, and Y. F. Zhong, "A novel ant colony algorithm for assembly sequence planning," *Int. J. Adv. Manuf. Technol.*, vol. 25, nos. 11–12, pp. 1137–1143, 2005.
- [18] S. Malik, "Performance comparison between ant algorithm and modified ant algorithm," *Int. J. Adv. Comput. Sci. Appl.*, vol. 1, no. 4, pp. 42–46, Oct. 2010.
- [19] S. M. McGovern and S. M. Gupta, "Ant colony optimization for disassembly sequencing with multiple objectives," *Int. J. Adv. Manuf. Technol.*, vol. 30, no. 5, pp. 481–496, 2006.
- [20] B. Adenso-Díaz, S. García-Carbajal, and S. Lozano, "An efficient GRASP algorithm for disassembly sequence planning," *OR Spectr.*, vol. 29, no. 3, pp. 535–549, 2007.
- [21] B. Adenso-Díaz, S. García-Carbajal, and S. M. Gupta, "A path-relinking approach for a bi-criteria disassembly sequencing problem," *Comput. Oper. Res.*, vol. 35, no. 12, pp. 3989–3997, 2008.
- [22] C. Andrés, S. Lozano, and B. Adenso-Díaz, "Disassembly sequence planning in a disassembly cell context," *Robot. Comput. Integr. Manuf.*, vol. 23, no. 6, pp. 690–695, 2007.
- [23] M. Gao, M. C. Zhou, and R. J. Caudill, "Integration of disassembly leveling and bin assignment for demanufacturing automation," *IEEE Trans. Robot. Autom.*, vol. 18, no. 6, pp. 867–874, Dec. 2002.
- [24] A. Gungor and S. M. Gupta, "Disassembly sequence planning for products with defective parts in product recovery," *Comput. Ind. Eng.*, vol. 35, nos. 1–2, pp. 161–164, 1998.
- [25] L. Brennan, S. M. Gupta, and K. N. Taleb, "Operations planning issues in an assembly/disassembly environment," *Int. J. Oper. Prod. Manag.*, vol. 14, no. 9, pp. 57–67, 1994.
- [26] G. D. Tian, J. W. Chu, and T. Qiang, "Influence factor analysis and prediction models for component removal time in manufacturing," *Proc. Inst. Mech. Eng. B J. Eng. Manuf.*, vol. 227, no. 10, pp. 1533–1540, 2013.
- [27] G. D. Tian, M. C. Zhou, and J. W. Chu, "A chance constrained programming approach to determine the optimal disassembly sequence," *IEEE Trans. Autom. Sci. Eng.*, vol. 10, no. 4, pp. 1004–1013, Oct. 2013.
- [28] G. D. Tian, M. C. Zhou, J. W. Chu, and Y. M. Liu, "Probability evaluation models of product disassembly cost subject to random removal time and different removal labor cost," *IEEE Trans. Autom. Sci. Eng.*, vol. 9, no. 2, pp. 288–295, Apr. 2012.
- [29] Y. Tang, M. C. Zhou, E. Zussman, and R. J. Caudill, "Disassembly modeling, planning, and applications," *J. Manuf. Syst.*, vol. 21, no. 2, pp. 200–217, 2002.
- [30] Y. Tang, M. C. Zhou, and R. J. Caudill, "An integrated approach to disassembly planning and demanufacturing operation," *IEEE Trans. Robot. Autom.*, vol. 17, no. 6, pp. 773–784, Dec. 2001.
- [31] E. Zussman and M. C. Zhou, "A methodology for modeling and adaptive planning of disassembly processes," *IEEE Trans. Robot. Autom.*, vol. 15, no. 1, pp. 190–194, Feb. 1999.
- [32] H.-J. Kim, R. Harms, and G. Seliger, "Automatic control sequence generation for a hybrid disassembly system," *IEEE Trans. Autom. Sci. Eng.*, vol. 4, no. 2, pp. 194–205, Apr. 2007.
- [33] I. Aguinaga, D. Borro, and L. Matey, "Parallel RRT-based path planning for selective disassembly planning," *Int. J. Adv. Manuf. Technol.*, vol. 36, nos. 11–12, pp. 1221–1233, 2008.
- [34] Y. P. Ren, G. D. Tian, F. Zhao, D. Y. Yu, and C. Y. Zhang, "Selective cooperative disassembly planning based on multi-objective discrete artificial bee colony algorithm," *Eng. Appl. Artif. Intel.*, vol. 64, pp. 415–431, Sep. 2017.
- [35] M. Gao, M. C. Zhou, and Y. Tang, "Intelligent decision making in disassembly process based on fuzzy reasoning Petri nets," *IEEE Trans. Syst., Man, Cybern. B. Cybern.*, vol. 34, no. 5, pp. 2029–2034, Oct. 2004.
- [36] X. H. Liu, G. L. Peng, X. M. Liu, and Y. F. Hou, "Disassembly sequence planning approach for product virtual maintenance based on improved max–min ant system," *Int. J. Adv. Manuf. Technol.*, vol. 59, nos. 5–8, pp. 829–839, 2012.
- [37] Y. Tang and M. C. Zhou, "A systematic approach to design and operation of disassembly lines," *IEEE Trans. Autom. Sci. Eng.*, vol. 3, no. 3, pp. 324–329, Jul. 2006.
- [38] Y. Tang, "Learning-based disassembly process planner for uncertainty management," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 39, no. 1, pp. 134–143, Jan. 2009.
- [39] X. W. Guo, S. X. Liu, M. C. Zhou, and G. D. Tian, "Disassembly sequence optimization for large-scale products with multiresource constraints using scatter search and Petri nets," *IEEE Trans. Cybern.*, vol. 46, no. 11, pp. 2435–2446, Nov. 2016.
- [40] S. A. Reveliotis, "Uncertainty management in optimal disassembly planning through learning-based strategies," *IEE Trans.*, vol. 30, no. 6, pp. 645–658, 2007.
- [41] C. B. Kalayci and S. M. Gupta, "Ant colony optimization for sequence-dependent disassembly line balancing problem," *J. Manuf. Technol. Manag.*, vol. 24, no. 3, pp. 413–427, 2013.
- [42] C. Bierwirth and D. C. Mattfeld, "Production scheduling and rescheduling with genetic algorithms," *Evol. Comput.*, vol. 7, no. 1, pp. 1–17, 1999.
- [43] B. Naderi and R. Ruiz, "A scatter search algorithm for the distributed permutation flowshop scheduling problem," *Eur. J. Oper. Res.*, vol. 239, no. 2, pp. 323–334, 2014.
- [44] X. W. Guo, S. X. Liu, M. C. Zhou, and G. D. Tian, "Dual-objective program and scatter search for the optimization of disassembly sequences subject to multiresource constraints," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 3, pp. 1091–1103, Jul. 2018.
- [45] L. P. Kerkhove and M. Vanhoucke, "A parallel multi-objective scatter search for optimising incentive contract design in projects," *Eur. J. Oper. Res.*, vol. 261, no. 3, pp. 1066–1084, 2017.
- [46] E. Kongar and S. M. Gupta, "Disassembly sequencing using genetic algorithm," *Int. J. Adv. Manuf. Technol.*, vol. 30, nos. 5–6, pp. 497–506, 2006.
- [47] J. Li, J. Zhang, C. Jiang, and M. C. Zhou, "Composite particle swarm optimizer with historical memory for function optimization," *IEEE Trans. Cybern.*, vol. 45, no. 10, pp. 2350–2363, Oct. 2015.
- [48] B. Liu, K. Huang, J. Li, and M. C. Zhou, "An incremental and distributed inference method for large-scale ontologies based on MapReduce paradigm," *IEEE Trans. Cybern.*, vol. 45, no. 1, pp. 53–64, Jan. 2015.
- [49] Y. P. Fu, H. F. Wang, G. D. Tian, Z. W. Li, and H. S. Hu, "Two-agent stochastic flow shop deteriorating scheduling via a hybrid multi-objective evolutionary algorithm," *J. Int. Manuf.*, vol. 30, no. 6, pp. 1–16, Jan. 2018. [Online]. Available: <https://doi.org/10.1007/s10845-017-1385-4>
- [50] V. A. Shim, K. C. Tan, and H. J. Tang, "Adaptive memetic computing for evolutionary multiobjective optimization," *IEEE Trans. Cybern.*, vol. 45, no. 4, pp. 610–621, Apr. 2015.
- [51] M. A. González, C. R. Vela, and R. Varela, "Scatter search with path relinking for the flexible job shop scheduling problem," *Eur. J. Oper. Res.*, vol. 245, no. 1, pp. 35–45, 2015.



Xiwang Guo received the B.S. degree in computer science and technology from the Shenyang Institute of Engineering, Shenyang, China, in 2006, the M.S. degree in aeronautics and astronautics manufacturing engineering from Shenyang Aerospace University, Shenyang, in 2009, and the Ph.D. degree in system engineering from Northeastern University, Shenyang, in 2015.

He is currently an Associate Professor with the College of Computer and Communication Engineering, Liaoning Shihua University, Fushun, China. From 2016 to 2018, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ, USA. He has authored over 30 technical papers in journals and conference proceedings, including the IEEE TRANSACTIONS ON CYBERNETICS, the IEEE TRANSACTIONS ON SYSTEM, MAN AND CYBERNETICS: SYSTEMS, the IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, and the IEEE/CAA JOURNAL OF AUTOMATICA SINICA. His current research interests include Petri nets, remanufacturing, recycling and reuse of automotive, and intelligent optimization algorithm.



Mengchu Zhou (S'88–M'90–SM'93–F'03) received the B.S. degree in control engineering from the Nanjing University of Science and Technology, Nanjing, China, in 1983, the M.S. degree in automatic control from the Beijing Institute of Technology, Beijing, China, in 1986, and the Ph.D. degree in computer and systems engineering from Rensselaer Polytechnic Institute, Troy, NY, USA, in 1990.

He joined the New Jersey Institute of Technology, Newark, NJ, USA, in 1990, where he is currently a Distinguished Professor of electrical and computer engineering. He has over 800 publications, including 12 books, 460 journal papers (over 360 in IEEE TRANSACTIONS), and 28 book chapters. He holds 12 patents and several pending ones. His current research interests include intelligent automation, Petri nets, Internet of Things, Web service, workflow, big data, transportation, and energy systems.

Dr. Zhou was a recipient of the National Science Foundation (NSF)'s Research Initiation Award, the CIM University-LEAD Award from Society of Manufacturing Engineers, the Perlis Research Award and Fenster Innovation in Engineering Education Award from NJIT, the Humboldt Research Award for U.S. Senior Scientists from Alexander von Humboldt Foundation, the Leadership Award and Academic Achievement Award from Chinese Association for Science and Technology, USA, the Asian American Achievement Award from Asian American Heritage Council of New Jersey, and the Outstanding Contributions Award, Distinguished Lectureship, the Franklin V. Taylor Memorial Award, and the Norbert Wiener Award from IEEE SMC Society, and the Distinguished Service Award from IEEE Robotics and Automation Society. He was invited to lecture in Australia, Canada, China, France, Germany, Hong Kong, Italy, Japan, South Korea, Mexico, Qatar, Saudi Arabia, Singapore, Taiwan, and U.S. and served as a plenary/keynote speaker for many conferences. He is the Founding Editor of IEEE Press Book Series on Systems Science and Engineering and the Editor-in-Chief of the IEEE/CAA JOURNAL OF AUTOMATICA SINICA. He served as an Associate Editor for the IEEE TRANSACTIONS ON ROBOTICS AND AUTOMATION, the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING, the IEEE TRANSACTIONS ON SYSTEMS, MAN AND CYBERNETICS: SYSTEMS, and the IEEE TRANSACTIONS ON INDUSTRIAL INFORMATICS, and an Editor for the IEEE TRANSACTIONS ON AUTOMATION SCIENCE AND ENGINEERING. He served as a Guest-Editor for many journals, including the IEEE INTERNET OF THINGS JOURNAL, the IEEE TRANSACTIONS ON INDUSTRIAL ELECTRONICS, and the IEEE TRANSACTIONS ON SEMICONDUCTOR MANUFACTURING. He is also an Associate Editor of the IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, the IEEE INTERNET OF THINGS JOURNAL, and the *Frontiers of Information Technology and Electronic Engineering*. He was the General Chair of IEEE Conference on Automation Science and Engineering, Washington, DC, USA, in 2008, the General Co-Chair of 2003 IEEE International Conference on System, Man, and Cybernetics (SMC), Washington DC, USA, in 2003, the Founding General Co-Chair of 2004 IEEE International Conference on Networking, Sensing and Control, Taipei, in 2004, and the General Chair of 2006 IEEE International Conference on Networking, Sensing and Control, Fort Lauderdale, FL, USA, in 2006. He was the Program Chair of 2010 IEEE International Conference on Mechatronics and Automation, Xi'an, China, in 2010, 1998 and 2001 IEEE International Conference on SMC, and 1997 IEEE International Conference on Emerging Technologies and Factory Automation. He organized and chaired over 100 technical sessions and served on program committees for many conferences. He has led or participated in over 50 research and education projects with total budget over \$12M, funded by NSF, Department of Defense, NIST, New Jersey Science and Technology Commission, and industry. He is the Founding Co-Chair of Enterprise Information Systems Technical Committee (TC) and Environmental Sensing, Networking, and Decision-Making TC of IEEE SMC Society. He is also a VP for Conferences and Meetings of IEEE SMC Society. He has been among most highly cited scholars for years and ranked top one in the field of engineering worldwide in 2012 by Web of Science/Thomson Reuters and currently Clarivate Analytics. He is a Life Member of the Chinese Association for Science and Technology, USA, and served as its President in 1999. He is a fellow of the International Federation of Automatic Control, the American Association for the Advancement of Science, and the Chinese Association of Automation.



Shixin Liu received the B.S. degree in mechanical engineering from Southwest Jiaotong University, Chengdu, China, in 1990, and the M.S. and Ph.D. degrees in systems engineering from Northeastern University, Shenyang, China, in 1993 and 2000, respectively.

He is currently a Professor with the College of Information Science and Engineering, Northeastern University. He has over 100 publications, including one book. His current research interests include intelligent optimization algorithm, project management, and theory and method of planning and scheduling.



Liang Qi (S'16–M'18) received the B.S. degree in information and computing science and the M.S. degree in computer software and theory from the Shandong University of Science and Technology, Qingdao, China, in 2009 and 2012, respectively, and the Ph.D. degree in computer software and theory from Tongji University, Shanghai, China, in 2017.

He is currently a Lecturer of computer science and technology with the Shandong University of Science and Technology. From 2015 to 2017, he was a visiting student with the Department of Electrical and Computer Engineering, New Jersey Institute of Technology, Newark, NJ, USA. He has authored over 20 technical papers in journals and conference proceedings, including the IEEE TRANSACTIONS ON SYSTEM, MAN AND CYBERNETICS: SYSTEMS, the IEEE TRANSACTIONS ON INTELLIGENT TRANSPORTATION SYSTEMS, and the IEEE/CAA JOURNAL OF AUTOMATICA SINICA. His current research interests include Petri nets, discrete event systems, and optimization algorithms.

Dr. Qi was a recipient of the Best Student Paper Award-Finalist in the 15th IEEE International Conference on Networking, Sensing and Control.